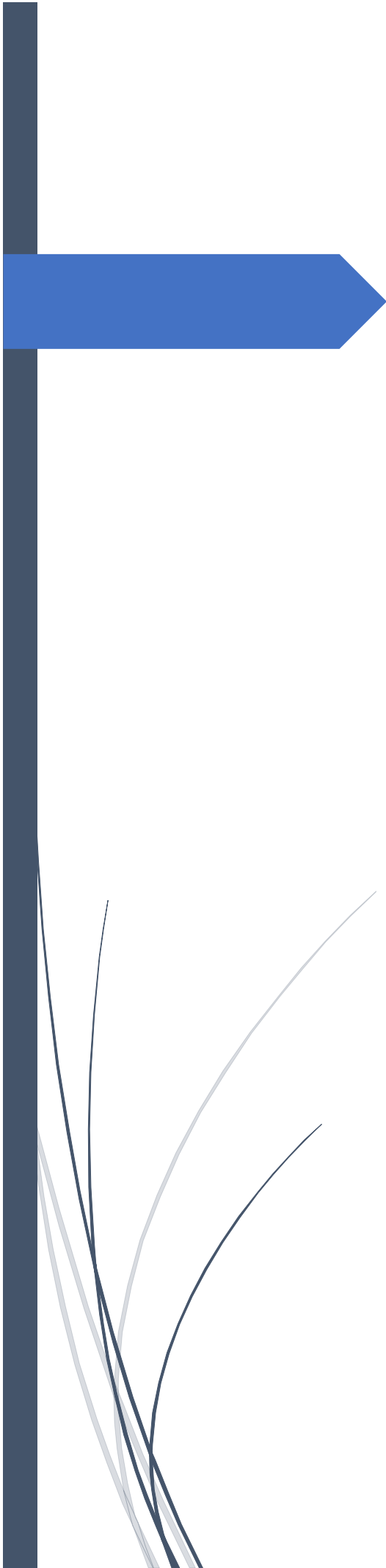




Inventario Manager

Sistema de gestión de inventario
para Android

Luis Jesus Rodriguez Rabadan Morote

IES Gregorio Prieto

PFG Desarrollo de Aplicaciones Multiplataforma

2025-2026

<https://github.com/luisj2/InventarioManager>

Tabla de contenido

1. Introducción	2
Finalidad y objetivos.....	2
Requisitos	2
2. Análisis del sistema actual.....	2
Aportación del proyecto	2
Principales aportaciones de Inventario Manager	3
Comparación con aplicaciones existentes en Google Play	3
3. Solución propuesta.....	4
3.1 Tecnologías existentes	4
3.2 Elección y justificación	4
4. Planificación temporal.....	5
Diseño de la solución.....	5
Desarrollo	5
Fase de pruebas	5
5. Documentación del diseño e implementación	5
Arquitectura	5
CASOS DE USO.....	7
Manual de usuario.....	21
6. Bibliografía y fuentes de información	40

1. Introducción

Finalidad y objetivos

El presente proyecto consiste en el desarrollo de una aplicación móvil orientada a la gestión de inventario basada en zonas. Estas zonas pueden ser de uso individual o compartido entre varios usuarios, permitiendo una organización flexible de los artículos.

La aplicación permite gestionar artículos dentro de distintas zonas, registrar movimientos de inventario entre ellas y compartir dichas zonas con otros usuarios, facilitando el trabajo colaborativo.

El objetivo principal es proporcionar una herramienta sencilla, intuitiva y accesible que permita gestionar inventarios de forma eficiente desde un dispositivo móvil, mejorando el control de los artículos almacenados y optimizando su organización y localización.

Requisitos

La aplicación desarrollada cumple con los siguientes requisitos funcionales:

- Registro e inicio de sesión de usuarios.
- Creación y gestión de zonas de almacenamiento, tanto en local como en la nube.
- Añadir, modificar y eliminar artículos dentro de cada zona.
- Registro de movimientos de artículos entre distintas zonas.
- Compartición de zonas entre múltiples usuarios para facilitar la colaboración.

2. Análisis del sistema actual

- Aplicaciones similares.
- Apps de inventario en Android.
- Excel / hojas de cálculo.
- Sistemas ERP.

Aportación del proyecto

Actualmente existen diversas herramientas para la gestión de inventario, como hojas de cálculo (Excel o Google Sheets) o sistemas ERP empresariales. Sin embargo, estas soluciones presentan ciertas limitaciones:

- **Hojas de cálculo:** requieren una gestión manual y son propensas a errores humanos, especialmente cuando se manejan grandes volúmenes de productos o múltiples ubicaciones.
- **Sistemas ERP:** suelen ser complejos, costosos y están orientados a empresas de mayor tamaño, lo que dificulta su adopción por pequeñas y medianas empresas o usuarios individuales.

- **Aplicaciones móviles existentes:** muchas permiten registrar productos y generar reportes básicos, pero no ofrecen mecanismos efectivos de colaboración entre varios usuarios ni control de permisos por roles.

En este contexto, el proyecto “**Inventario Manager**” propone una solución innovadora y adaptada a las necesidades modernas de gestión de inventario, aportando las siguientes ventajas:

Principales aportaciones de Inventario Manager

1. **Gestión jerárquica de inventario por zonas:**
La aplicación organiza los productos en **zonas padres e hijas**, permitiendo estructurar el inventario de manera clara y adaptable a almacenes físicos o virtuales complejos.
2. **Soporte para almacenamiento local y en la nube:**
 - La gestión local permite trabajar sin conexión a internet, ideal para entornos con conectividad limitada.
 - La sincronización en la nube mantiene los datos actualizados entre varios dispositivos y usuarios.
3. **Colaboración entre usuarios:**
 - Inventario Manager permite **compartir zonas de inventario** con otros usuarios mediante solicitudes.
 - Los usuarios pueden aceptar o rechazar la inclusión de nuevos miembros, garantizando control sobre la información compartida.
 - Cada miembro tiene permisos claros, y solo los propietarios pueden eliminar zonas completas.
4. **Registro y autenticación de usuarios:**
 - La aplicación integra un sistema de **login y registro**, asegurando que cada usuario acceda únicamente a las zonas que le corresponden.
 - Esto mejora la trazabilidad de las acciones y garantiza la seguridad de los datos.
5. **Gestión de movimientos y cambios en tiempo real:**
 - Los movimientos de inventario (entradas, salidas o cambios de ubicación) se registran y sincronizan automáticamente.
 - Esto evita errores de duplicidad o pérdida de información.

Comparación con aplicaciones existentes en Google Play

Inventario Manager destaca frente a otras aplicaciones de inventario por ofrecer una gestión jerárquica de zonas, permitiendo organizar ubicaciones principales y secundarias. Además, incorpora sincronización en la nube, colaboración multiusuario y un sistema de registro e inicio de sesión, funcionalidades que suelen estar ausentes o limitadas en aplicaciones similares.

También permite registrar automáticamente los movimientos y cambios de inventario, mejorando el control y la trazabilidad de los productos. Por último, cuenta con una interfaz moderna desarrollada con Jetpack Compose, proporcionando una experiencia de usuario más intuitiva y actualizada.

3. Solución propuesta

3.1 Tecnologías existentes

Para el desarrollo de esta aplicación multiplataforma, se analizaron diversas alternativas tecnológicas presentes en el sector informático:

- **Lenguajes de programación:** Se consideró el uso de Java, al ser el lenguaje tradicional para Android, frente a Kotlin, que es el lenguaje recomendado actualmente por Google por su seguridad y concisión.
- **Gestión de datos locales:** Se evaluó el uso de SQLite nativo frente a la librería de persistencia Room, que actúa como una capa de abstracción más robusta y eficiente.
- **Servicios en la nube y backend:** Se sopesó la creación de un backend propio (usando Node.js o PHP con bases de datos MySQL) frente al uso de plataformas BaaS (Backend as a Service) como Firebase.

3.2 Elección y justificación

Tras realizar el estudio de viabilidad técnica, se han seleccionado las siguientes tecnologías por su eficiencia y adecuación a los requerimientos del proyecto:

- **Kotlin:** Elegí Kotlin porque es más cómodo y moderno para el desarrollo en Android, ofreciendo las mismas funcionalidades que Java de forma más concisa y segura. Se utiliza para toda la lógica de negocio, manejo de interfaces y conexión con bases de datos, aprovechando su sintaxis moderna y sus características de seguridad de nulls.
- **Room (Persistencia local):** Room se emplea para almacenar datos localmente y garantizar que la aplicación funcione en modo offline. Permite mapear los objetos de la base de datos a objetos Kotlin de manera sencilla, asegurando la integridad de los datos locales y facilitando consultas eficientes sin necesidad de depender de la conexión a Internet.
- **Firebase Auth:** Se utiliza para gestionar de manera segura la autenticación de los usuarios, permitiendo el acceso mediante correo/contraseña o proveedores externos como Google. Esto simplifica la gestión de seguridad y asegura una experiencia de login confiable.
- **Firebase Firestore:** Se eligió como base de datos en la nube por su capacidad de sincronización en tiempo real entre dispositivos y su escalabilidad automática. Firestore guarda y sincroniza los datos principales de los usuarios, de modo que cualquier cambio realizado en un dispositivo se refleje instantáneamente en los demás.

4. Planificación temporal

Diseño de la solución

Durante la fase inicial del proyecto se estableció una arquitectura base y se definieron los requisitos funcionales principales de la aplicación. A medida que avanzaba el desarrollo, la estructura del sistema y la organización de las pantallas se adaptaron según las necesidades prácticas, realizando ajustes en el modelo de datos y en la interfaz para mejorar la funcionalidad y la experiencia de usuario.

Desarrollo

Posteriormente, se procedió a la implementación de la aplicación utilizando Kotlin y Android Studio. En esta fase se desarrollaron las funcionalidades principales:

- Gestión de usuarios.
- Creación de zonas.
- Gestión de artículos.
- Registro de movimientos.
- Compartición de zonas.

Fase de pruebas

Una vez implementadas las funcionalidades principales, se realizaron diversas pruebas para comprobar el correcto funcionamiento de la aplicación y detectar posibles errores.

5. Documentación del diseño e implementación

Arquitectura

Presentation (UI)

La capa Presentation es la encargada de mostrar la información al usuario y gestionar su interacción con la aplicación. Se comunica exclusivamente con la capa Domain para ejecutar operaciones.

Incluye:

- **Pantallas (UI):** Representan las distintas vistas de la aplicación. Cada pantalla dispone de su propio ViewModel y sigue un enfoque basado en gestión de estado mediante State, Events y Effects:
 - **ViewModel:** Gestiona la lógica de presentación y actúa como intermediario entre la UI y la capa Domain. Procesa eventos y actualiza el estado de la interfaz.
 - **State:** Representa el estado de la pantalla en cada momento. Se define mediante una data class que contiene toda la información necesaria para

renderizar la UI. La interfaz observa este estado y se actualiza automáticamente cuando cambia.

- **Events:** Son las acciones del usuario (como pulsar botones o introducir datos). Se envían al ViewModel para ser procesadas.
- **Effects:** Representan acciones puntuales que no forman parte del estado, como navegación, mostrar mensajes (Toast, Snackbar) o diálogos.
- **Navegación:** Gestiona el flujo entre pantallas.
- **Componentes reutilizables de UI:** Elementos comunes para mantener consistencia y reducir duplicación de código.
- **Temas:** Definen apariencia visual (colores, tipografías, estilos).

Domain (Lógica de negocio / Casos de uso)

Contiene la lógica de negocio de la aplicación y es independiente de cualquier tecnología o framework externo. Actúa como intermediaria entre Presentation y Data.

Incluye:

- **Casos de uso (Use Cases):** Operaciones principales como CrearZona, MoverArtículo o CompartirZona.
- **Repositorios (interfaces):** Definen operaciones para acceder a los datos sin especificar implementación, desacoplando la lógica de negocio de las fuentes de datos.
- **Modelos de dominio:** Representan las entidades principales de la aplicación.

Data (Persistencia)

Gestiona el acceso a los datos de la aplicación, tanto local como remoto.

Incluye:

- **Fuentes de datos:**
 - Room para persistencia local.
 - Firestore como base de datos en la nube.
- **Implementación de repositorios:** Contienen las implementaciones concretas de las interfaces definidas en Domain.
- **Mappers:** Transforman datos entre modelos de las fuentes y modelos de dominio.

DI (Inyección de dependencias)

Se utiliza para inicializar repositorios, use cases y ViewModels de la aplicación.

Utils

Utilidades y aplicaciones de vistas comunes.

CASOS DE USO

```

@luisj2
sealed class SuspendResult<out T> {
    @luisj2
    data object Idle : SuspendResult<Nothing>()
    @luisj2
    data object Loading : SuspendResult<Nothing>()
    @luisj2
    data class Success<out T>(val data: T) : SuspendResult<T>()
    @luisj2
    data class Error(val message: String) : SuspendResult<Nothing>()
}

```

- Todas las operaciones asíncronas que interactúan con Room o Firestore devuelven un **SuspendResult<T>**.

- Para todas las operaciones de acceso a datos, se utiliza la clase sellada **SuspendResult**, que permite encapsular el estado de la operación y su resultado. Esto facilita la gestión de errores y la interacción con la capa de presentación, evitando excepciones no controladas y simplificando la observación del estado de la operación desde los ViewModel.
- Esto permite **unificar la gestión de estados**, manejando:
 - Idle → cuando aún no se ha ejecutado la operación
 - Loading → cuando está en curso
 - Success(data) → cuando la operación termina correctamente y devuelve un resultado
 - Error(message) → cuando ocurre un error con su respectivo mensaje

```

protected suspend fun <T> executeRoomOperation(block: suspend () -> T): SuspendResult<T> {
    return withContext(Dispatchers.IO) {
        try {
            SuspendResult.Success(block()) ^withContext
        } catch (e: Exception) {
            SuspendResult.Error(RoomExceptionHandler.handle(e)) ^withContext
        }
    }
}

```



```

luisj2
protected suspend fun <T> executeFirestoreOperation(block: suspend () → T): SuspendResult<T> {
    return withContext(Dispatchers.IO) {
        try {
            SuspendResult.Success(block()) ^withContext
        } catch (e: Exception) {
            SuspendResult.Error(FirestoreExceptionHandler.handle(e)) ^withContext
        }
    }
}

```

Estas funciones permiten que todos los métodos de acceso a datos devuelvan un `SuspendResult`, centralizando la gestión de errores y resultados. Al usar `withContext(Dispatchers.IO)`, las operaciones se ejecutan en un hilo separado, evitando bloquear la UI y manteniendo la aplicación responsiva. Los estados `Idle`, `Loading`, `Success` y `Error` permiten que la capa de presentación maneje de forma consistente la carga, los errores y los datos recibidos.

OBTENER TODAS LAS ZONAS DE PARA MOSTRAR

```

val firestoreResult = zoneFirestoreRepository.getUserZones(userId)
firestoreResult.onSuccess { zones →
    zones.forEach { zoneFirestore →
        val zoneId = zoneFirestore.id ?: return@forEach
        val fullZoneResult = zoneFirestoreRepository.getZoneById(zoneId)
        val articlesResult = zoneFirestoreRepository.getAllArticleList(zoneId)

        fullZoneResult.onSuccess { fullZone →
            articlesResult.onSuccess { articles →
                val zone = fullZone.toDomain().copy(
                    articleList = articles.map { it.toDomain() }
                )
                allZones.add(zone)
            }.onError { errorMessage →
                return SuspendResult.Error("Error obteniendo artículos: $errorMessage")
            }
        }.onError { errorMessage →
            return SuspendResult.Error("Error obteniendo zona completa: $errorMessage")
        }
    }
}
}.onError { errorMessage →
    return SuspendResult.Error("Error obteniendo zonas de Firestore: $errorMessage")
}

// Room
val roomResult = zoneRoomRepository.getAllZonesFull(userId)
roomResult.onSuccess { zones →
    val roomZones = zones.mapNotNull { it?.toDomain() }
    allZones.addAll(roomZones)
}.onError { errorMessage →
    return SuspendResult.Error("Error obteniendo zonas de Room: $errorMessage")
}

return SuspendResult.Success(allZones)

```

Primero en la parte de firestore obtengo todas las zonas que tiene el usuario y después digamos que construyo el objeto con la parte de los artículos como de el objeto de la zona obtenida de firestore, y la parte de room con el método ya obtengo los datos completos de la zona y los uno los resultados en una lista conjunta de los dos y es lo que devuelvo en el método el SuspendResult de la lista conjunta

FIRESTORE

```
override suspend fun getUserZones(userId: String): SuspendResult<List<ZoneFirestore>> {  
    return executeFirestoreOperation {  
  
        val ownerQuery = getZoneCollection()  
            .whereEqualTo(FIRESTORE_ZONE_OWNER_FIELD, userId) Query  
            .get() Task<QuerySnapshot!>  
            .await() QuerySnapshot!  
            .toObjects(ZoneFirestore::class.java)  
  
        val memberQuery = getZoneCollection()  
            .whereArrayContains(FIRESTORE_ZONE_MEMBERS_FIELD, userId) Query  
            .get() Task<QuerySnapshot!>  
            .await() QuerySnapshot!  
            .toObjects(ZoneFirestore::class.java)  
  
        val combined = (ownerQuery + memberQuery)  
            .distinctBy { it.id }  
  
        combined ^executeFirestoreOperation  
    }  
}
```

El usuario tiene dos tipos de zonas compartidas, las zonas que ha creado él y las zonas que no ha creado pero a las que se ha unido como miembro, en este método obtiene las dos y las une en una lista que es lo que devuelve el método

```
override suspend fun getZoneById(zoneId: String): SuspendResult<ZoneFirestore> {  
    return executeFirestoreOperation {  
        getZoneDocumentRef(zoneId).get().await().toObject( valueType = ZoneFirestore::class.java)  
            ?: throw IllegalArgumentException("No se encontró la zona con id: $zoneId")  
    }  
}
```

Obtengo la zona de firestore a partir de su Id

```
override suspend fun getAllArticleList(zoneId: String): SuspendResult<List<ArticleFirestore>> {  
    return executeFirestoreOperation {  
        getArticleCollection(zoneId).get().await().toObjects( clazz = ArticleFirestore::class.java)  
    }  
}
```

Obtengo los artículos de una zona

ROOM

```
luisj2
suspend fun getAllZonesFull(userId : String): SuspendResult<List<ZoneWithArticlesAndMovements?>> {
    return executeRoomOperation {
        zoneDao.getAllZoneFull(userId)
    }
}
```

```
luisj2
@Transactional
@Query("SELECT * FROM Zone WHERE userId = :userId")
suspend fun getAllZoneFull(userId : String): List<ZoneWithArticlesAndMovements>
```

Esto mismo en Room se puede hacer con una transacción de una unión entre tablas

MOSTRAR TODA LA INFORMACION DE UNA ZONA EN ESPECÍFICO

```
class GetZoneById (
    private val zoneRoomRepository: ZoneRoomRepository,
    private val zoneFirestoreRepository: ZoneFirestoreRepository
){

    new *
    suspend operator fun invoke (
        zoneId : String,
        storageType: StorageType
    ) : SuspendResult<Zone>{
        return obtainZoneByIdInDatabase(zoneId, storageType)
    }

    new *
    private suspend fun obtainZoneByIdInDatabase(
        zoneId : String,
        storageType: StorageType
    ) : SuspendResult<Zone>{
        val notFoundMessage = "No se ha encontrado la zona"
        return when(storageType){
            StorageType.LOCAL → {
                val zoneIdRoom = zoneId.toLongOrNull() ?: return SuspendResult.Error(notFoundMessage)
                val zoneFull = zoneRoomRepository.getZoneFull(zoneIdRoom).getOrNull() ?: return SuspendResult.Error(notFoundMessage)
                SuspendResult.Success(zoneFull.toDomain())
            }
            StorageType.FIREBASE → {
                val zoneFull = zoneFirestoreRepository.getFullZoneById(zoneId).getOrNull() ?: return SuspendResult.Error(notFoundMessage)
                SuspendResult.Success(zoneFull.toDomain())
            }
        }
    }
}
```

Este es el UseCase que pasara el objeto Zona y que mas tarde se mostrara en la vista, con el storageType detectamos que tipo de base de datos quiere el usuario obtener la zona de firebase o de room en este caso lo puse LOCAL, y los obtenemos con los respectivos métodos de su repositorio

ROOM

```
@Transaction
@Query("SELECT * FROM Zone WHERE userId = :userId")
suspend fun getAllZoneFull(userId : String): List<ZoneWithArticlesAndMovements>
```

Obtengo toda la información de la zona incluidos los movimientos y artículos de esta

FIRESTORE

```
suspend fun getFullZoneById(zoneId: String): SuspendResult<ZoneFullFirestore>
```

Obtengo la información completa con movimientos y artículos desde firestore

MOVER ARTICULOS ENTRE ZONAS

```
private suspend fun updateOrDeleteArticles(
    zoneId : String,
    articleId: String,
    quantityToRemove: Int,
    movement: ArticleMovement,
    storageType: StorageType
): SuspendResult<Boolean> {
    return when (storageType) {
        StorageType.LOCAL → {
            val articleRoomId = articleId.toLongOrNull()
            ?: return SuspendResult.Error("No se ha encontrado el artículo")
            val zoneIdRoom = zoneId.toLongOrNull() ?: return SuspendResult.Error("No se ha encontrado el identificador de la zona")

            zoneRoomRepository.updateOrDeleteArticlesAndMovements(
                zoneIdRoom,
                articleRoomId,
                quantityToRemove,
                movement.toEntity()
            )
        }
        StorageType.FIREBASE → {
            zoneFirestoreRepository.updateOrDeleteArticlesAndMovements(
                zoneId = zoneId,
                articleIdToRemove = articleId,
                quantityToRemove = quantityToRemove,
                movement = movement.toFirestore()
            )
        }
    }
}
```

Este método se encarga básicamente de actualizar la cantidad del artículo o eliminarlo según si se han eliminado toda la cantidad de dicho artículo, como siempre utilizamos el tipo de base de datos a usar con el objeto de storageType y en los respectivos métodos de los repositorios hacemos el resto

ROOM

```
luisj2
suspend fun updateOrDeleteArticlesAndMovements(
    zoneId : Long,
    articleId: Long,
    quantityToRemove: Int,
    movement: ArticleMovementsEntity
) : SuspendResult<Boolean>{
    return executeRoomOperation {
        zoneDao.updateOrDeleteArticlesAndMovements(
            zoneId, articleId, quantityToRemove, movement
        )
        true ^executeRoomOperation
    }
}
```

Simplemente aquí llamamos al método del dao que tiene la lógica que necesitamos

```

@Transaction
suspend fun updateOrDeleteArticlesAndMovements(
    zoneId : Long,
    articleId: Long,
    quantityToRemove: Int,
    movement: ArticleMovementsEntity
) {
    // 1 Obtener el count actual
    val currentArticleCount = getArticleCountById(zoneId, articleId)

    if (currentArticleCount != null) {
        // 2 Calcular la nueva cantidad
        val newCount = currentArticleCount - quantityToRemove

        // 3 Update o Delete según corresponda
        if (newCount > 0) {
            updateArticleCount(zoneId, articleId, newCount)
        } else {
            deleteArticleById(zoneId, articleId)
        }
    }

    // 4 Insertar el movimiento
    insertMovement(movement)
}

```

Desde este método del Dao primero obtenemos la cantidad que tiene el artículo que queremos modificar su cantidad o eliminar y obtenemos la nueva cantidad que va a tener dicho artículo, si es mayor que 0 modificamos la cantidad y si es 0 eliminamos el artículo de la zona y finalmente ponemos el movimiento que hemos hecho

FIRESTORE

```

override suspend fun updateOrDeleteArticlesAndMovements(
    zoneId: String,
    articleIdToRemove: String,
    quantityToRemove: Int,
    movement: ArticleMovementFirestore
): SuspendResult<Boolean> {
    return executeFirestoreOperation {
        // 1 Referencia al artículo dentro de la zona
        val articleRef = getArticleCollection(zoneId).document(documentPath = articleIdToRemove)

        // 2 Obtener snapshot actual
        val snapshot = articleRef.get().await()
        if (!snapshot.exists()) {
            throw IllegalStateException("El artículo no existe en la zona")
        }

        val article = snapshot.toObject(valueType = ArticleFirestore::class.java)
        ?: throw IllegalStateException("Error al mapear el artículo")

        val currentCount = article.count ?: 0
        val newCount = currentCount - quantityToRemove

        // 3 Update o delete según el nuevo count
        if (newCount > 0) {
            articleRef.update(FIRESTORE_ARTICLE_COUNT_FIELD, value = newCount).await()
        } else {
            articleRef.delete().await()
        }

        // 4 Insertar movimiento en la subcolección de la zona
        getMovementsCollection(zoneId)
            .document(documentPath = movement.id ?: "")
            .set(movement)
            .await()

        true ^executeFirestoreOperation
    }
}

```

Obtengo el artículo en concreto con el id de articleIdToRemove para eliminar y dependiendo de la cantidad de artículos que sea el newCount si es 0 se elimina y sino se actualiza a la nueva cantidad

```
private suspend fun updateOrInsertArticlesAndMovements(
    article: Article,
    zoneId : String,
    quantityToAdd: Int,
    movement: ArticleMovement,
    storageType: StorageType
): SuspendResult<Boolean> {
    return when (storageType) {
        StorageType.LOCAL → {
            val zoneIdRoom = zoneId.toLongOrNull() ?: return SuspendResult.Error("No se encontro la zona")
            val articleRoom = article.toZoneEntity().copy(
                count = quantityToAdd,
                zoneId = zoneIdRoom
            )
            val entityMovement = movement.toEntity().copy(zoneId = zoneIdRoom)

            zoneRoomRepository.upsertArticleAndMovement(zoneIdRoom, articleRoom, entityMovement)
        }
        StorageType.FIREBASE → {
            zoneFirestoreRepository.upsertArticleAndMovement(
                zoneId,
                quantityToAdd,
                article.toFirestore(),
                movement.toFirestore()
            )
        }
    }
}
```

Llamo al método en función de la base de datos local o de firebase para actualizar cantidad o insertar según si hay ya en la zona el mismo artículo o no

GUARDAR LOS DATOS EN LA BASE DE DATOS

```
new *
private suspend fun saveChangesInDatabase (
    zoneId : String,
    articlesToSave : List<Article>,
    movementsToSave : List<ArticleMovement>,
    storageType: StorageType
): SuspendResult<Boolean> {
    val articles = articlesToSave.map { it.copy(zoneId = zoneId) }
    val movements = movementsToSave.map { it.copy(zoneId = zoneId) }
    return when(storageType){
        StorageType.LOCAL → {
            val zoneIdRoom = zoneId.toLongOrNull() ?: return SuspendResult.Error("No se ha encontrado el identificador de la zona")
            val roomArticles = articles.map { it.toZoneEntity() }
            val roomMovements = movements.map { it.toEntity() }
            zoneRoomRepository.upsertArticlesAndMovements(zoneIdRoom, roomArticles, roomMovements)
        }
        StorageType.FIREBASE → {
            val firestoreArticles = articles.map { it.toFirestore() }
            val firestoreMovements = movements.map { it.toFirestore() }
            zoneFirestoreRepository.upsertArticleAndMovementLists(zoneId, firestoreArticles, firestoreMovements)
        }
    }
}
```

En la aplicación una vez que hemos hecho los cambios necesarios, nos saldrá un botón de guardar si le damos básicamente haremos este método que actualizara los cambios en los artículos y movimientos y según la zona en la que estemos local o compartida hará los cambios en dicha base de datos desde su repositorio

ROOM

```
luisj2
suspend fun upsertArticlesAndMovements(
    zoneId : Long,
    articles: List<ArticleZoneEntity>,
    movements: List<ArticleMovementsEntity>
) : SuspendResult<Boolean> {
    return executeRoomOperation {
        zoneDao.upsertArticlesAndMovements(zoneId, articles, movements)
        true ^executeRoomOperation
    }
}
```

```
luisj2
@Transactional
suspend fun upsertArticlesAndMovements(
    zoneId : Long,
    articles: List<ArticleZoneEntity>,
    movements: List<ArticleMovementsEntity>
) {
    insertOrUpdateArticleCountList(zoneId, articles)
    insertMovements(movements)
}
```

En estos métodos actualizamos la cantidad de los artículos y ponemos el movimiento del respectivo cambio que acabamos de hacer

METER LOS DATOS DE LOS ARTICULOS Y DE LOS MOVIMIENTOS PARA QUE SE MUESTREN PARA GUARDAR

```
new *
suspend operator fun invoke(
    articleSelectedList: List<ArticleSelectedEntity>,
    movementSelectedList: List<MovementSelectedEntity>
): SuspendResult<Boolean> {

    return articleRepo.insertOrUpdateArticles(articleSelectedList)
        .flatMap { articleOk →

            if (!articleOk) {
                SuspendResult.Error("No se pudieron insertar los artículos") ^flatMap
            } else {

                movementRepo.insertOrUpdateMovements(movementSelectedList)
                    .map { movementOk →
                        articleOk && movementOk
                    } ^flatMap
            }
        }
    }
}
```

Aquí lo que haríamos es cuando el usuario cambia algo de artículos en la zona se guardaría en una tabla de la base de datos tanto el artículo cambiado como el movimiento que se ha hecho para posteriormente si le da al botón de guardar que se guarde como he explicado previamente, así que esto es lo que hacemos guardamos en la base de datos los artículos que debemos de guardar o sea guardaríamos el método en el caso de que no exista o actualizaríamos la cantidad si ya está puesto para guardar y con el método de flatmap haríamos que digamos algo así como hacer estos dos métodos como uno solo y mostrar el resultado cuando los dos están hechos correctamente

```

@luisj2
suspend fun insertOrUpdateMovements(
    movements: List<MovementSelectedEntity>
): SuspendResult<Boolean> {

    return executeRoomOperation {
        if (movements.isEmpty()) throw Exception("No hay movimientos que añadir")

        movements.forEach { movement ->

            val existing = dao.getMovement(
                movement.screenId,
                movement.articleId,
                movement.zoneId
            )

            if (existing != null) {
                // Si existe actualizamos (ejemplo sumando cantidad)
                val updatedMovement = existing.copy(
                    count = existing.count + movement.count
                )

                dao.updateMovement(updatedMovement)
            } else {
                // Si no existe insertamos
                dao.insertMovementSelected(listOf(movement))
            }
        }

        true ^executeRoomOperation
    }
}

```

Este método añade la lista de movimientos a la tabla de la base de datos para guardar dichos movimientos,

REGISTRAR Y GUARDAR LOS DATOS DEL USUARIO REGISTRADO EN FIRESTORE

```

@luisj2
suspend operator fun invoke(
    user: UserFirestore,
    password: String
): ValidationResult {
    return authRepository.registerUser(user.email, password)
        .flatMap { uid ->
            userFirestoreRepository.insertUser(user.copy(uid = uid))
        }.toValidationResult()
}

```

Con este UseCase registramos el usuario en firebaseAuth y con el uid generado guardamos la información que quiero del usuario dentro de firestore con insertUser y usamos flatmap para hacerlo como una única operación y que devuelva el resultado de los dos métodos como uno solo

```
override suspend fun registerUser(email: String, password: String): SuspendResult<String> {  
    return executeAuthOperation {  
        val result = auth.createUserWithEmailAndPassword( p0 = email, p1 = password).await()  
        val uid = result.user?.uid ?: throw Exception("No se obtuvo UID")  
        auth.signOut()  
        uid ^executeAuthOperation  
    }  
}
```

Con este método registraríamos al usuarios en firebase auth

```
Usage  
override suspend fun insertUser(userFirestore: UserFirestore): SuspendResult<Boolean> =  
    executeFirestoreOperation {  
        saveUser(userFirestore)  
        true ^executeFirestoreOperation  
    }
```

```
private suspend fun saveUser(userFirestore: UserFirestore) {  
    getUserCollection().document( documentPath = userFirestore.uid).set(userFirestore).await()  
}
```

Lo guardamos en firestore la información del usuario

INICIAR SESIÓN

```
suspend operator fun invoke(  
    email: String,  
    password: String  
): ValidationResult {  
    return authLoginRepository.login(email, password)  
        .onSuccess { UserDataStore.saveUserId(it) }  
        .toValidationResult( errorMessageIfFalse: "No se ha podido completar el login")  
}
```

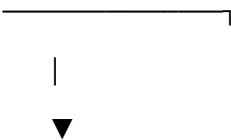
Iniciamos sesión en FirebaseAuth y guardamos en una base de datos local por así decir llamada DataStore los datos del usuarios para usarlos en los puntos de la aplicación donde me hagan falta

DIAGRAMA DE ROOM

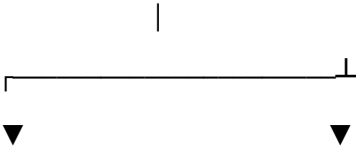


ORGANIZACIÓN DE COLLECCIONES DE FIRESTORE

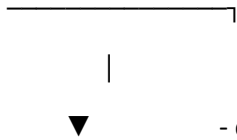
USER

- uid
 - email
 - userName
 - zonesIdList
- 

ZONE

- id
 - name
 - ownerId
 - membersId[]
 - parentIdList[]
 - childIdList[]
- 

ARTICLE

- id
 - name
 - category
 - zoneId
 - count
- 

ZONE REQUEST

- id
- zoneId
- zoneName
- requesterId
- receiverId
- createdAt

MOVEMENT

- id
- articleId
- userId
- zoneName
- count
- actionType
- date

Manual de usuario

[Crear cuenta](#)

Aquí tenemos la parte de autenticación en la que creamos la cuenta e iniciamos sesión en la aplicación solo tenemos que completar los campos con nuestra información de la parte de registro y después en la pantalla de inicio de sesión iniciamos sesión con el correo y la contraseña con la que nos registramos

Autenticación



Iniciar Sesión



Registrarse



¿Preparado para encontrar sin buscar?

Nombre de Usuario
Luis Jesus

Email
RabadanLjesus13@gmail.com

Contraseña
.....



Registrarse

Autenticación


Iniciar Sesión


Registrarse



Organiza tu inventario y olvida tu calvario

Email

RabadanLjesus13@gmail.com

Contraseña

.....



Iniciar Sesión

Entrar sin sesión

[¿Olvidaste tu contraseña? Pulsa aquí](#)

Crear zona

Aquí creamos la zona tan solo completamos el campo del nombre y le damos a siguiente y a crear la zona después nos llevara a la pantalla de zonas y la podremos ver creada



Crea tu Zona

Crea tu propia zona y asegúrate que el desorden no gane esta vez

Nombre de la zona

Zona padre

Zona padre

Ninguna

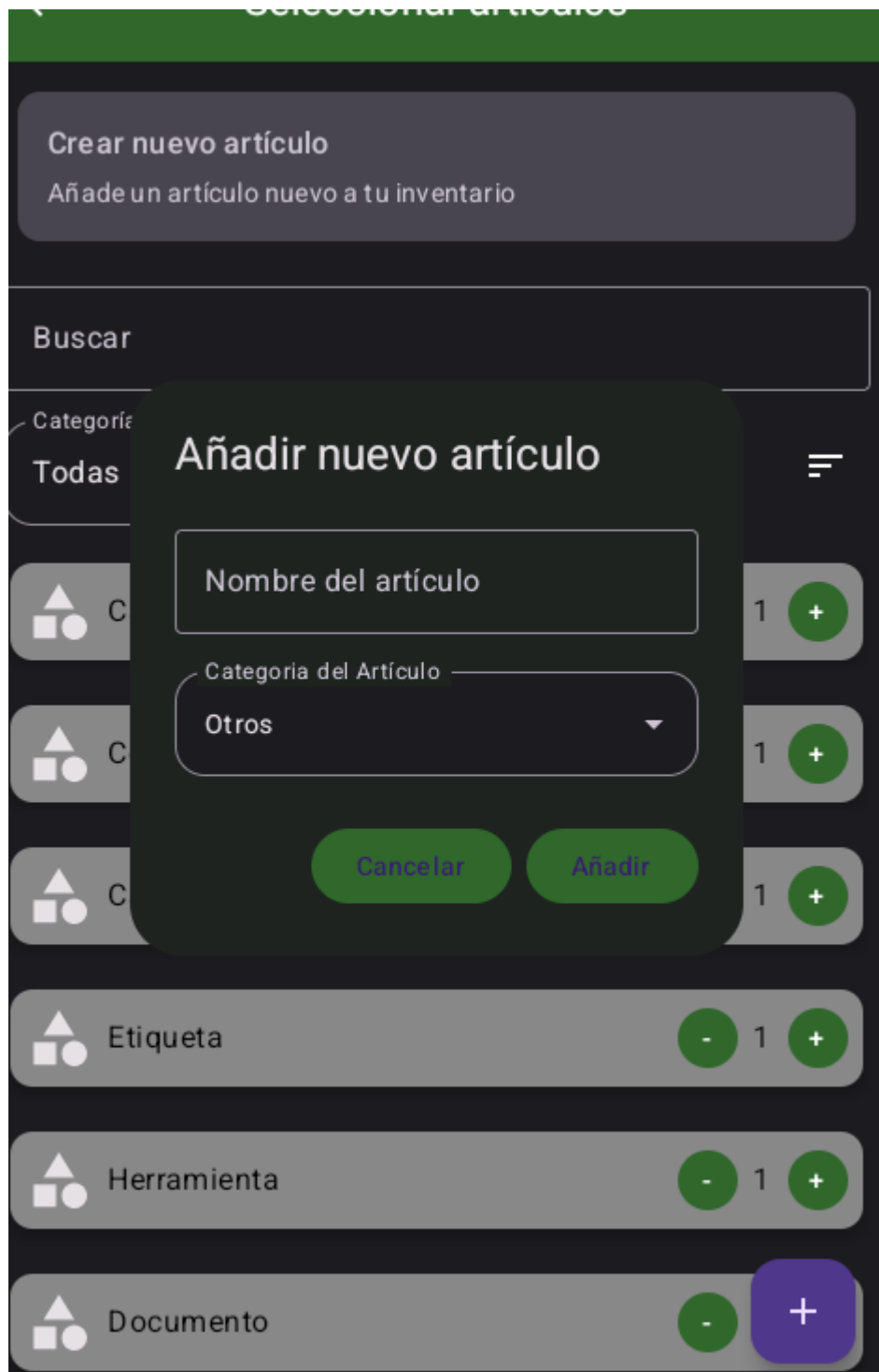


Añadir Artículos a la Zona que estamos creando

Seleccionamos los artículos que queremos con las cantidades de este y le damos al botón de debajo de +



Crear un nuevo artículo para añadir a las zonas



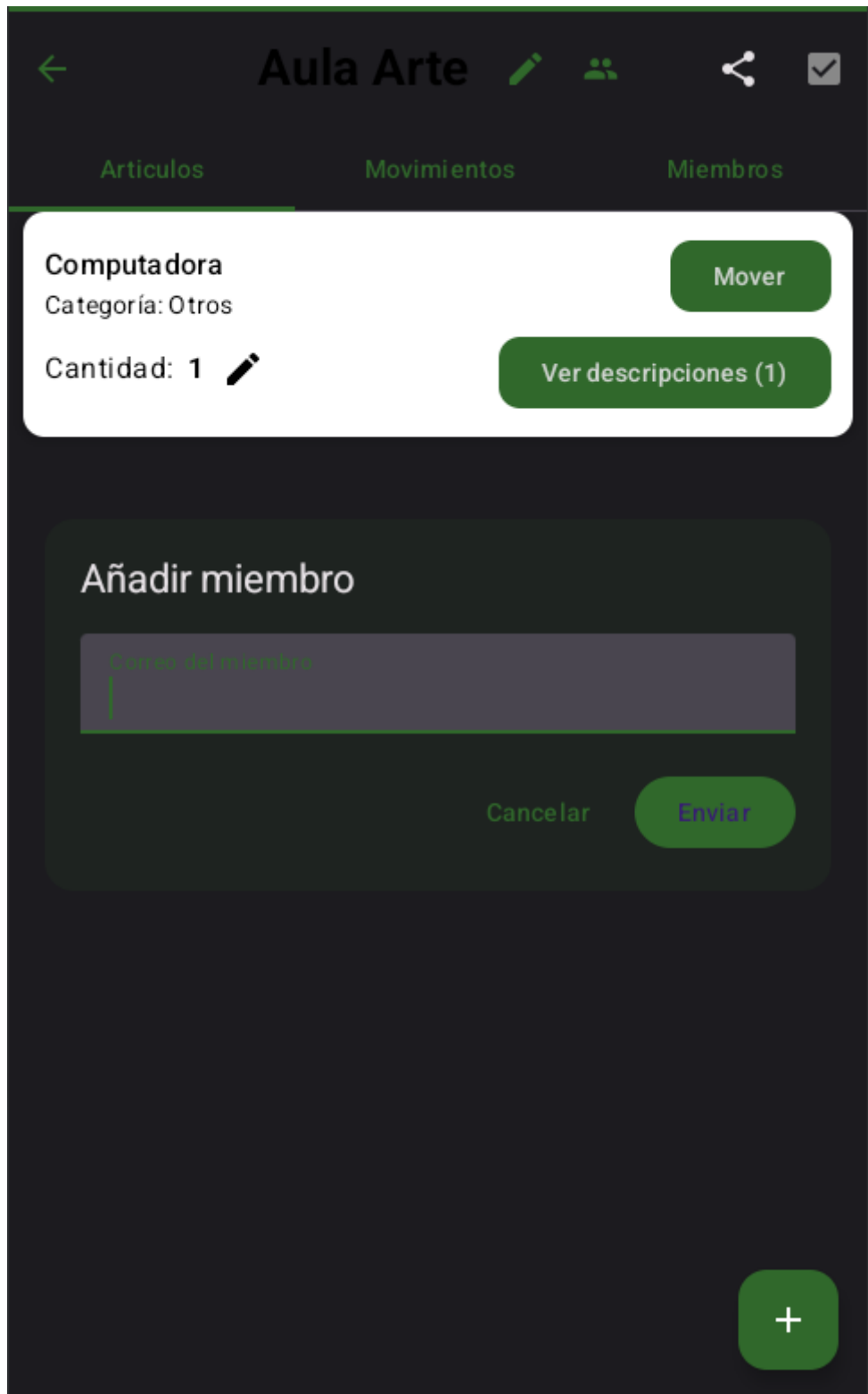
En el botón de arriba derecha podemos guardar los artículos agregados a la zona

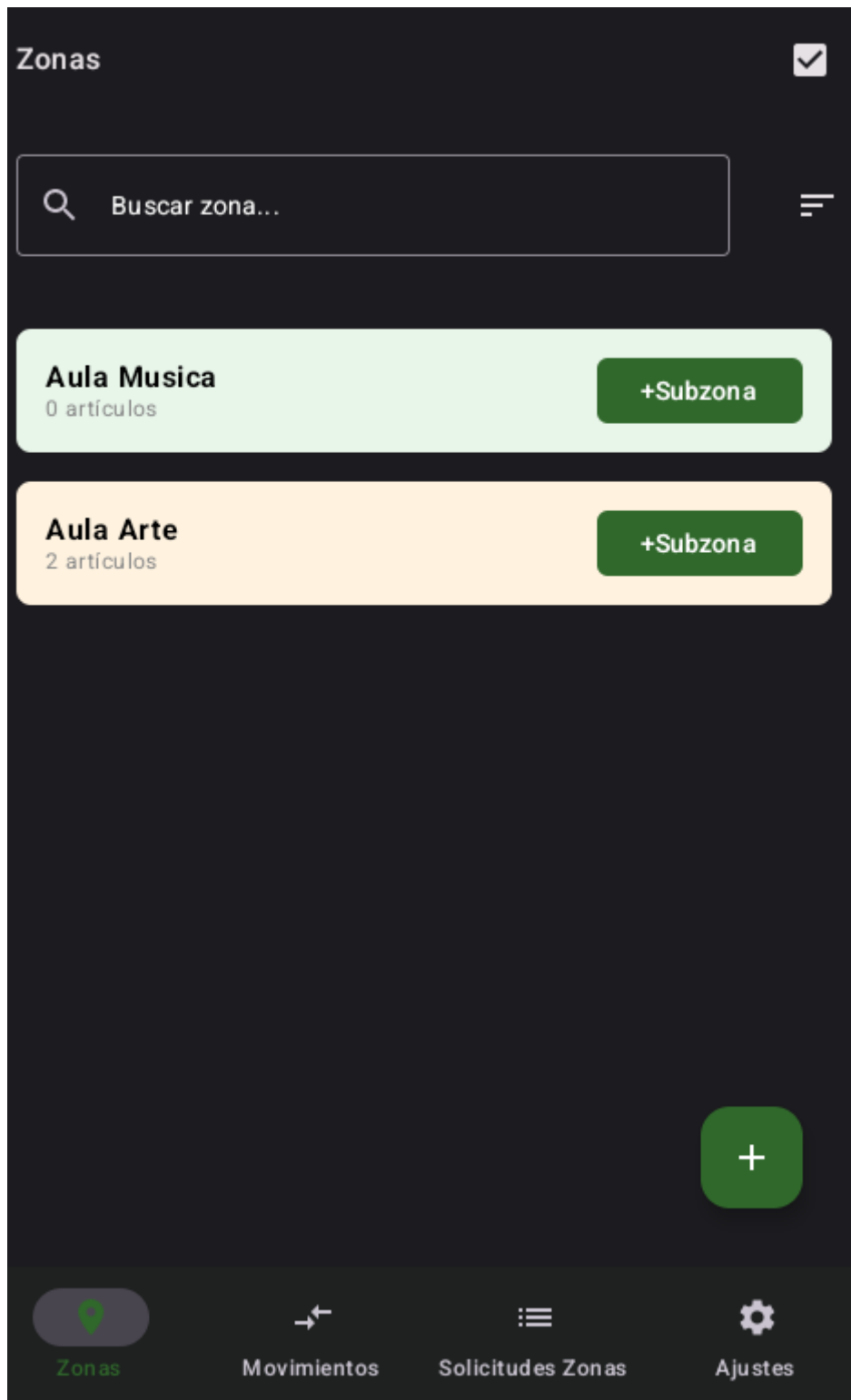


</

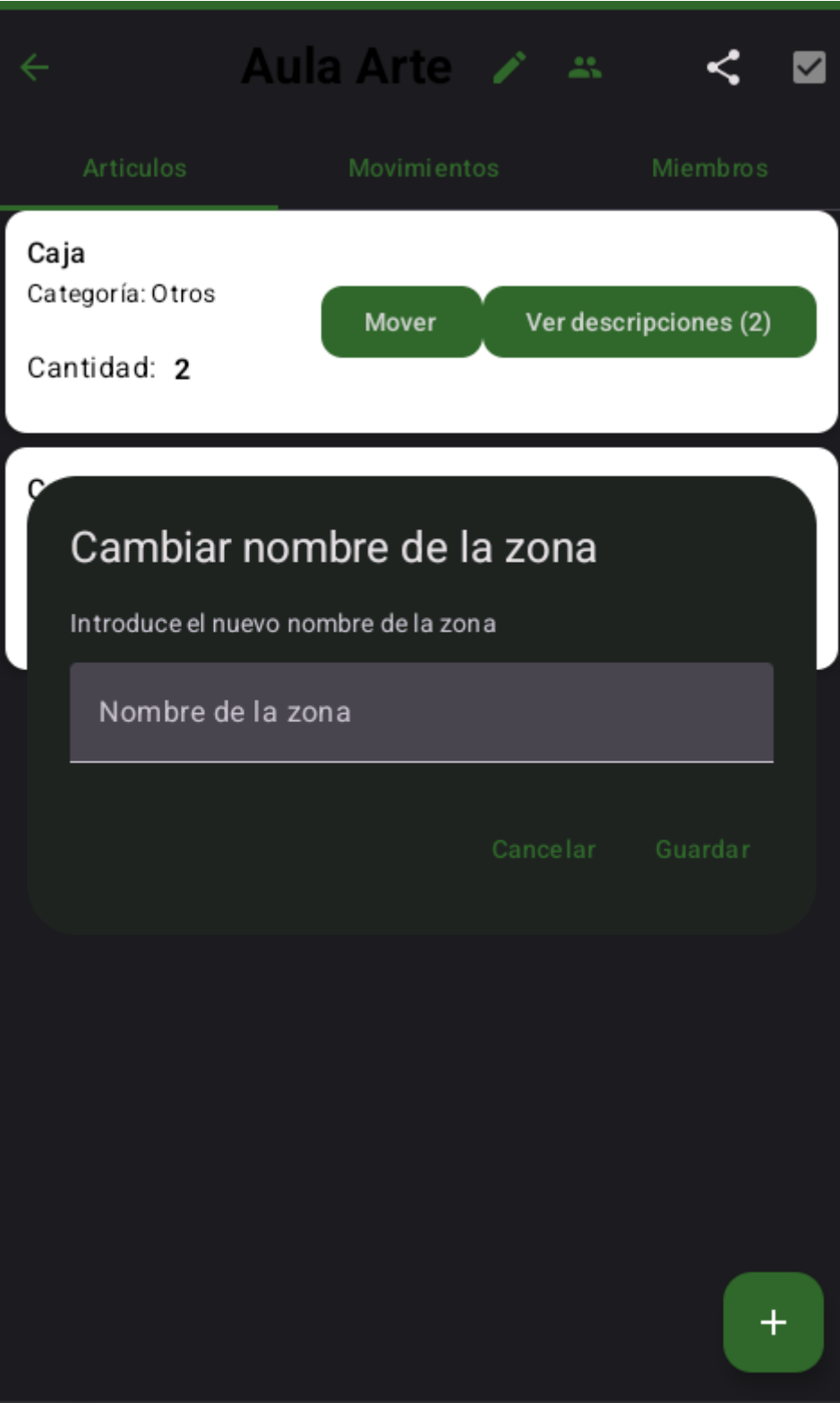
Dejando pulsado en una zona o dando en el botón de arriba derecha de selección activamos el modo de selección en donde podremos seleccionar los elementos para eliminar dándole al botón de basura después

Añadir miembro a la Zona

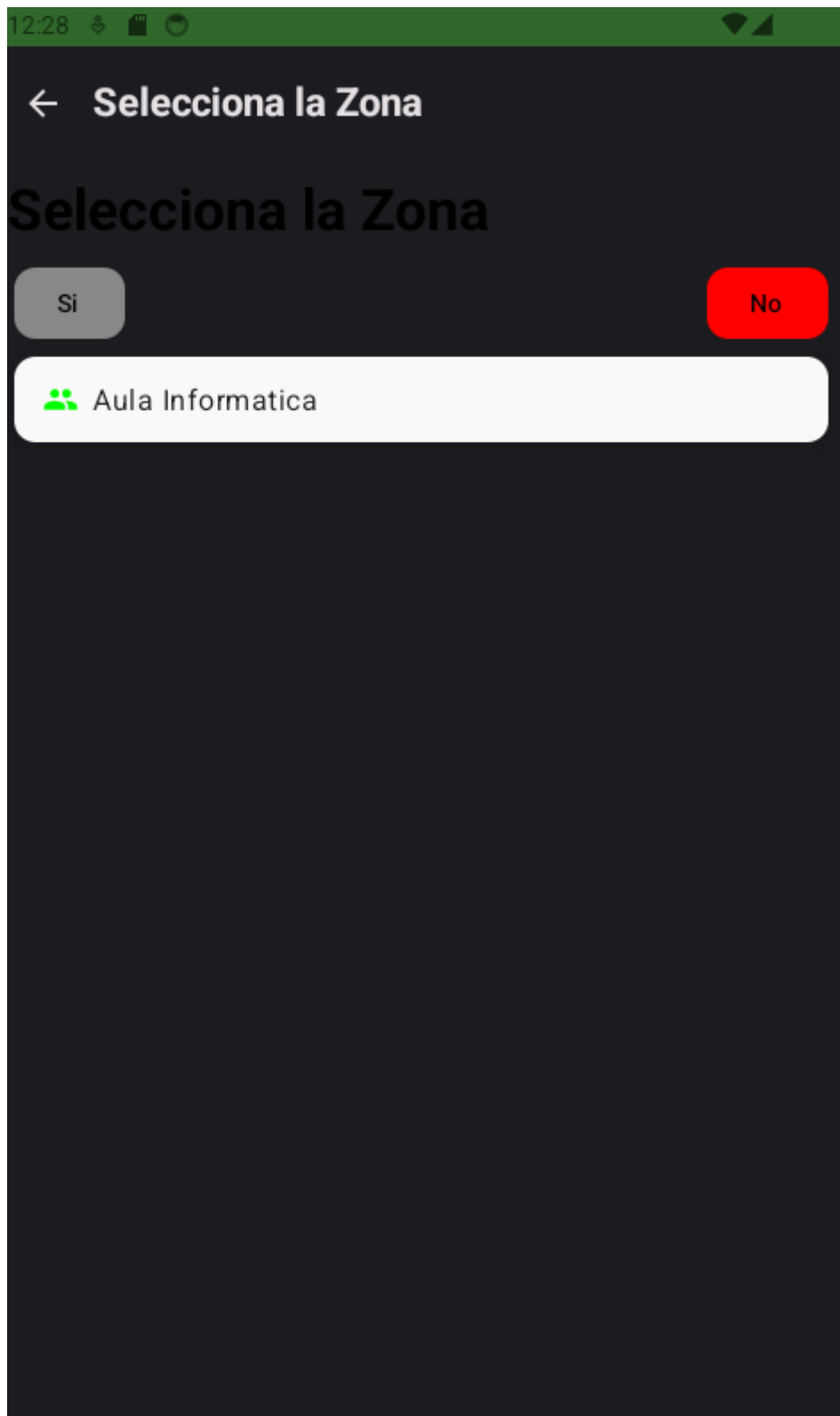




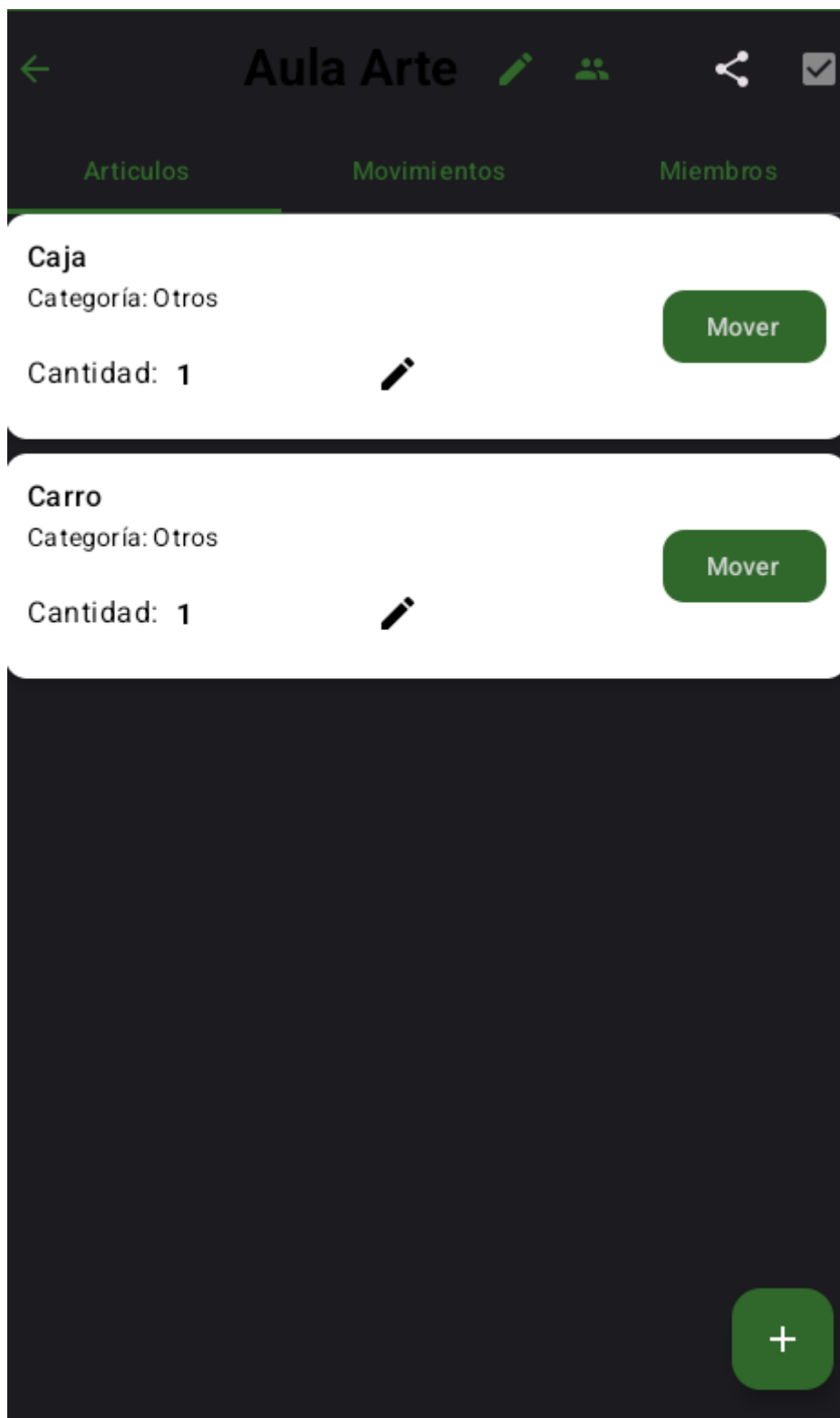
Aquí tenemos la pantalla principal de zonas en donde podremos ver nuestras zonas y a partir del color podremos diferenciar si es local o es compartida, si es gris es local y si es azul compartida, podremos crear una zona con el botón de mas abajo derecha de la pantalla o dándole al botón de SubZone



Mover artículo a otra Zona

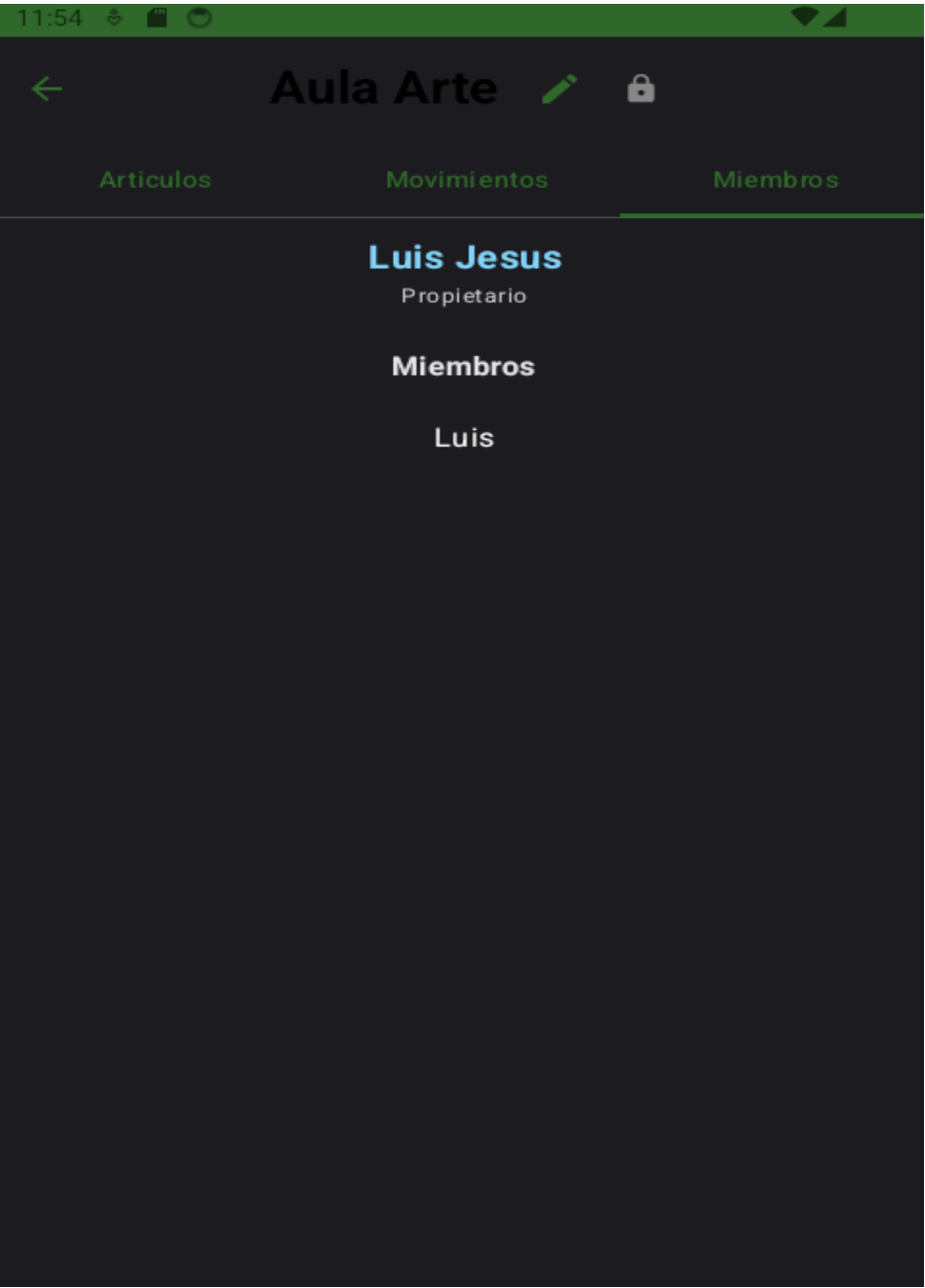


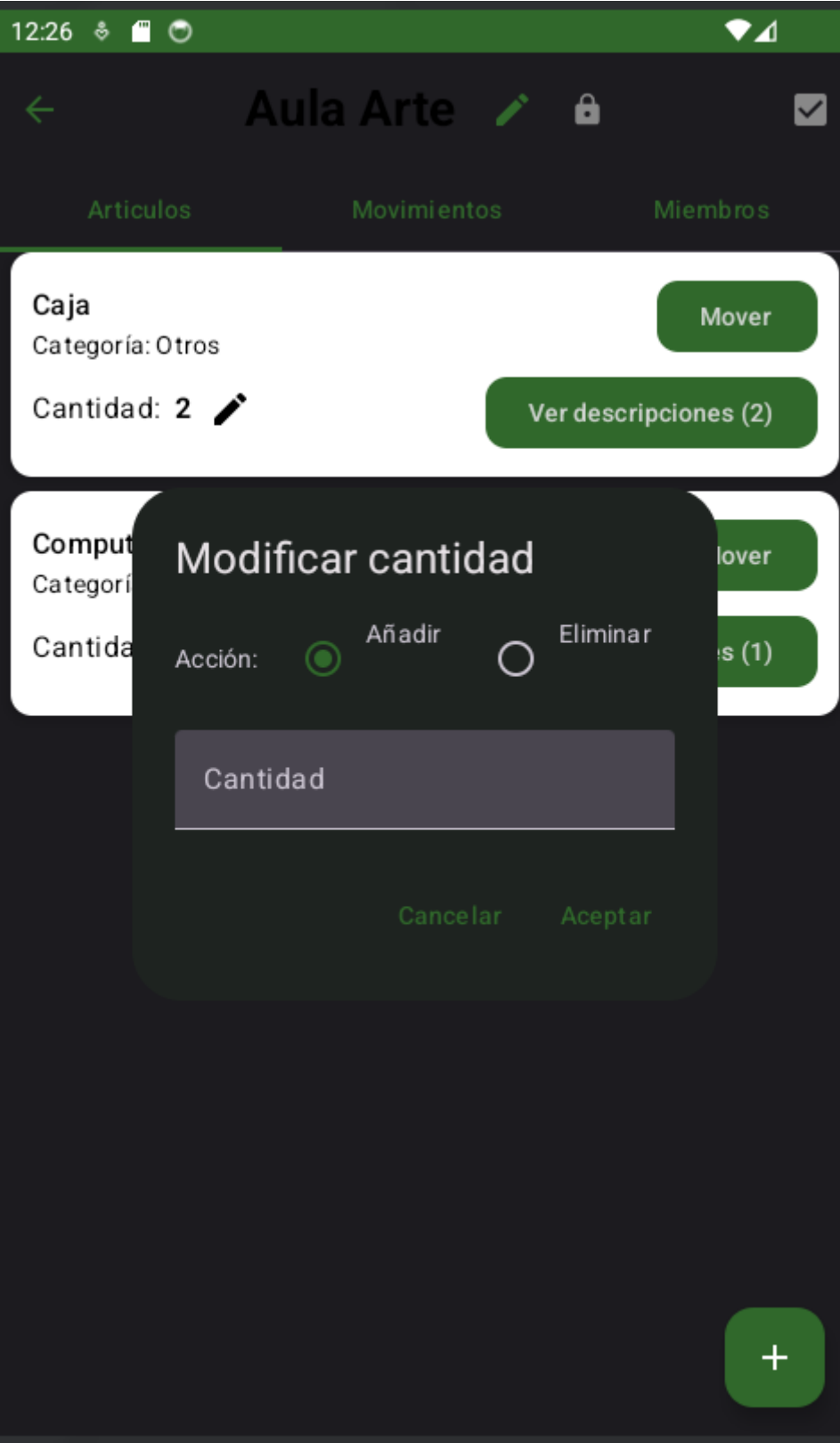
Aquí podemos
elegir a donde mover el articulo que hayamos elegido para moverá la otra zona



En la zona específica podremos ver los artículos que tenemos en dicha zona y moverlos a alguna otra z

Como se desde la perspectiva del Miembro







1 seleccionados



Artículos

Movimientos

Miembros

Caja

Categoría: Otros

Mover

Cantidad: 2 

Ver descripciones (2)

Computadora

Categoría: Otros

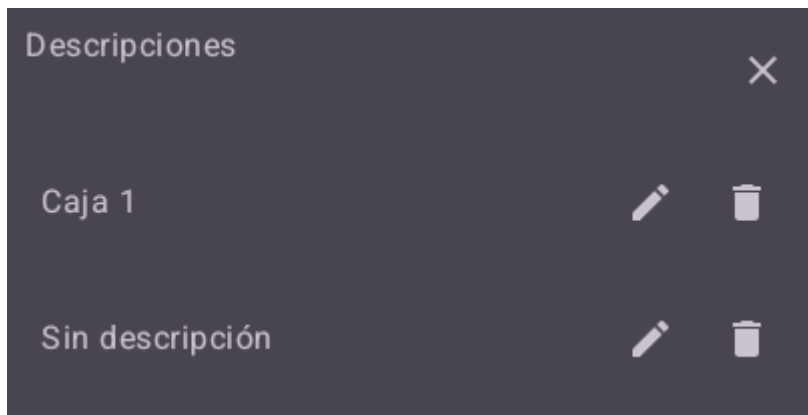
Mover

Cantidad: 1 

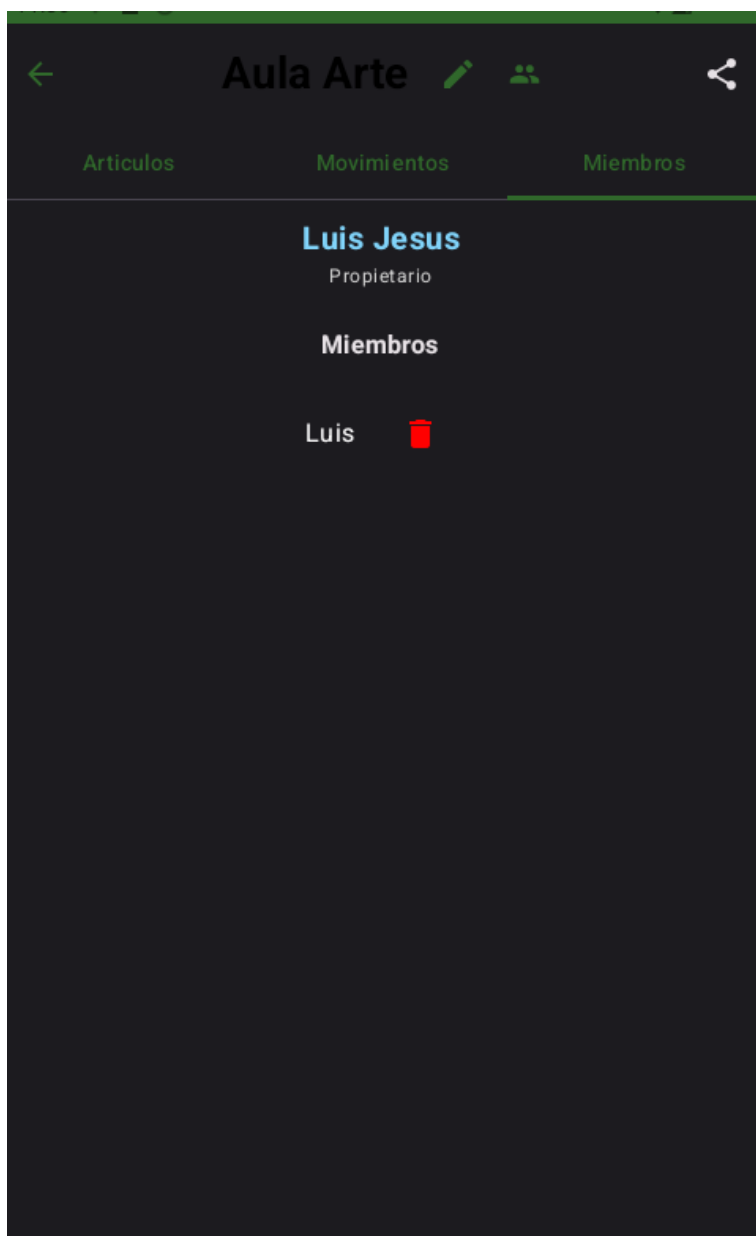
Ver descripciones (1)



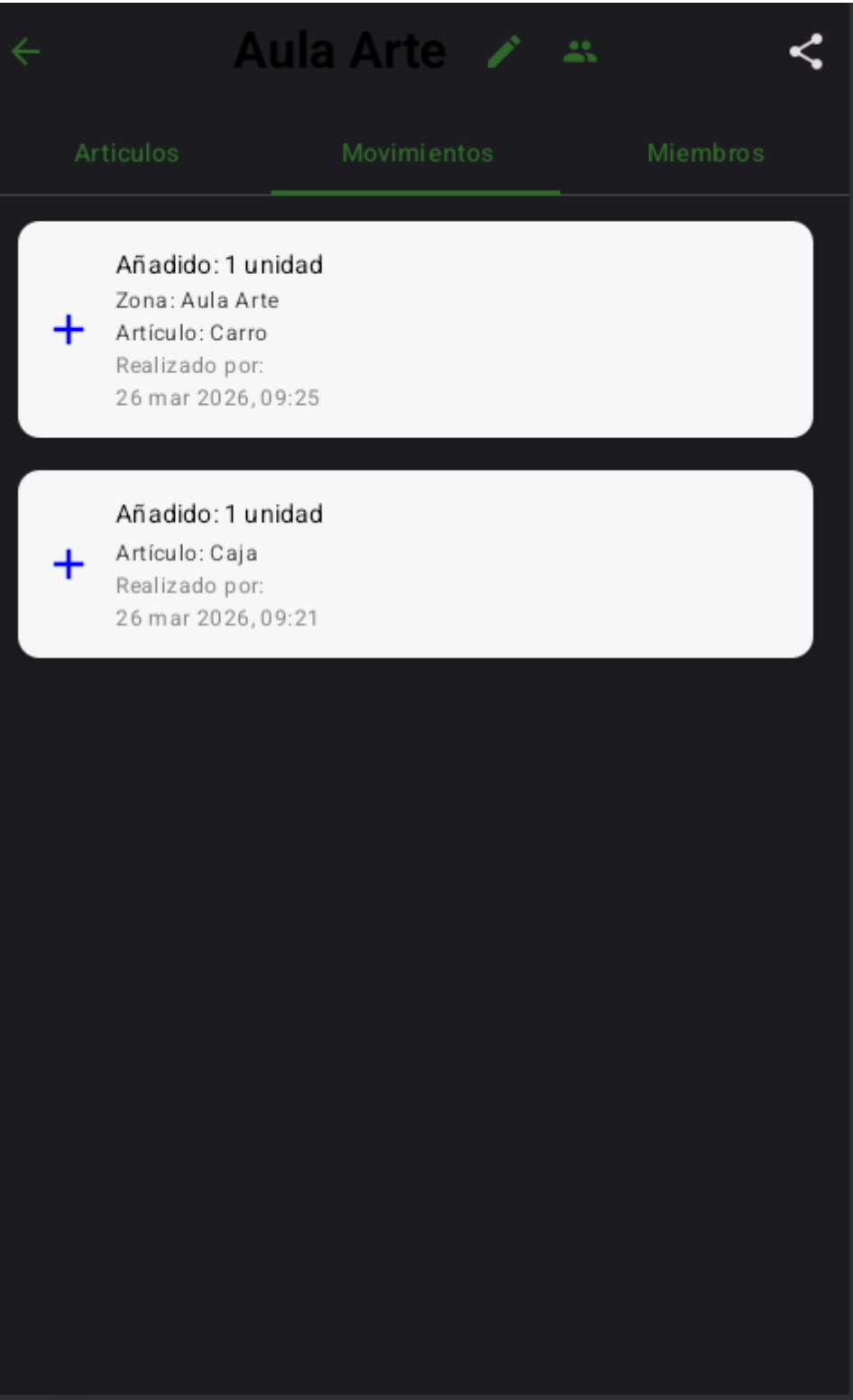
Ver descripciones y posibilidad de modificarlas



Miembros desde la perspectiva del propietario de la Zona

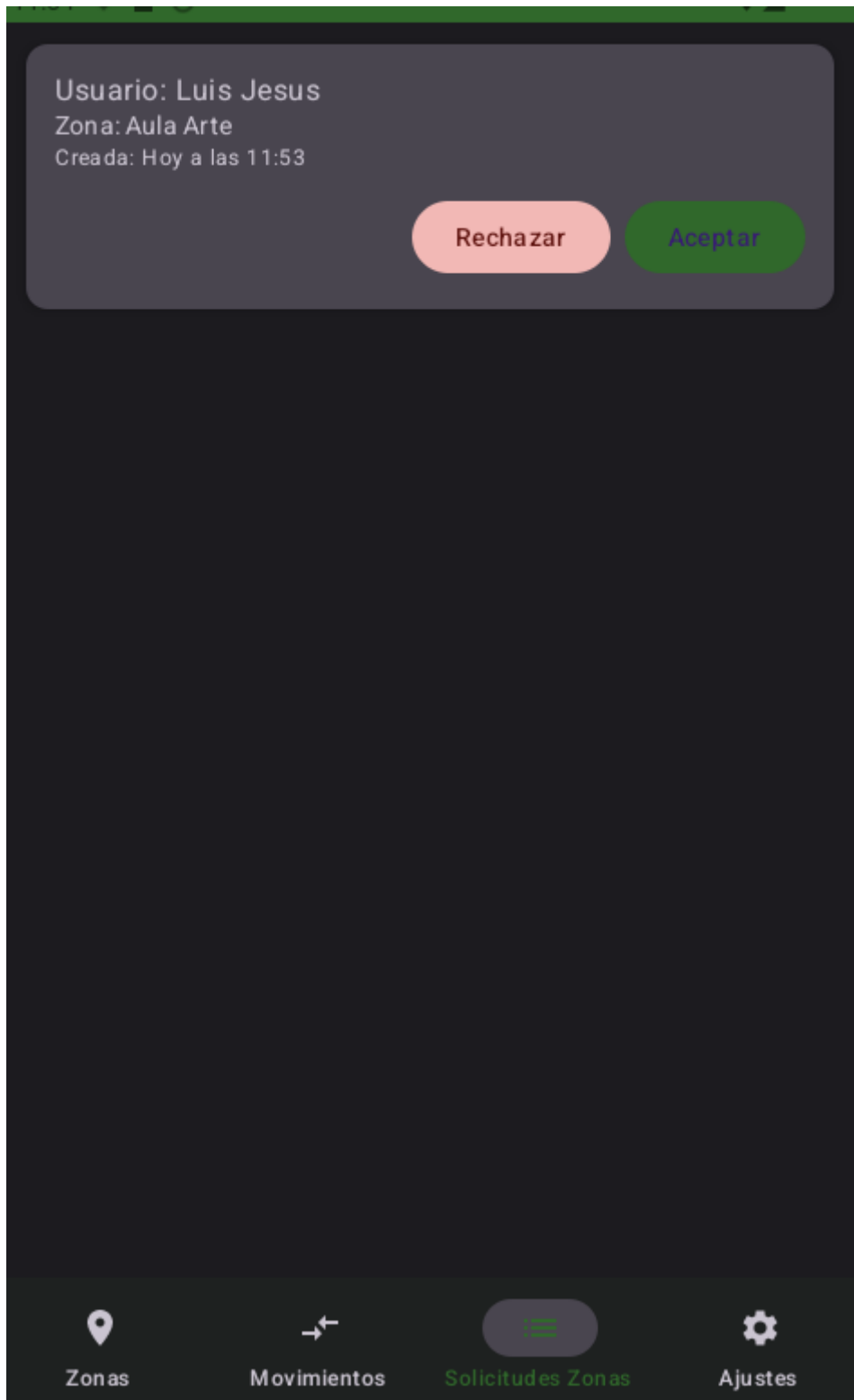


La parte de miembros en donde se gestiona quien es el propietario y dicho usuario puede eliminar a algun miembro de la zona y tambien puede compartir la zona a otros usuarios



Aquí podemos ver los movimientos que se han hecho dentro de la zona y datos de esta

SOLICITUDES ZONAS



Aquí podremos rechazar o aceptar solicitudes de zonas que nos hayan enviado para tenerlas en la lista también

6. Bibliografía y fuentes de información

- Android Developers – Android Studio y Kotlin: <https://developer.android.com>
- Kotlin Programming Language: <https://kotlinlang.org>
- Firebase Documentation: <https://firebase.google.com/docs>
- Room Persistence Library: <https://developer.android.com/topic/libraries/architecture/room>
- Jetpack Compose: <https://developer.android.com/jetpack/compose>

ML Kit – Barcode Scanning: <https://developers.google.com/ml-kit/vision/barcode-scanning>

Coil – Kotlin Image Loading: <https://coil-kt.github.io/coil/>

- Kotlinx Serialization: <https://github.com/Kotlin/kotlinx.serialization>
- Datastore Preferences: <https://developer.android.com/topic/libraries/architecture/datastore>
- Accompanist Swipe Refresh: <https://google.github.io/accompanist/swiperefresh/>
- IconDialog Library: <https://github.com/Maltaisn/IconDialog>
- Gson – JSON Serialization: <https://github.com/google/gson>